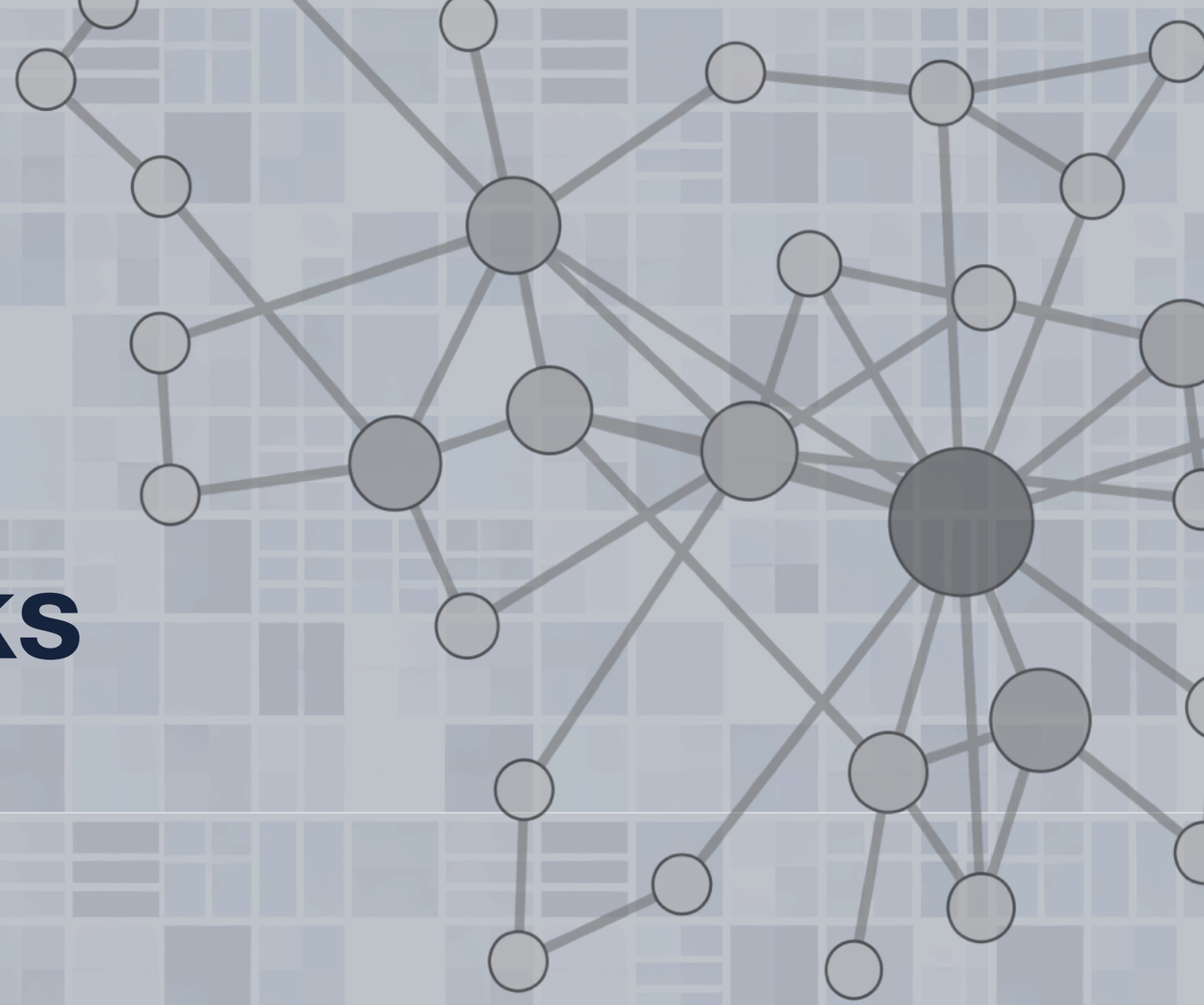
COMP30850

Creating Networks

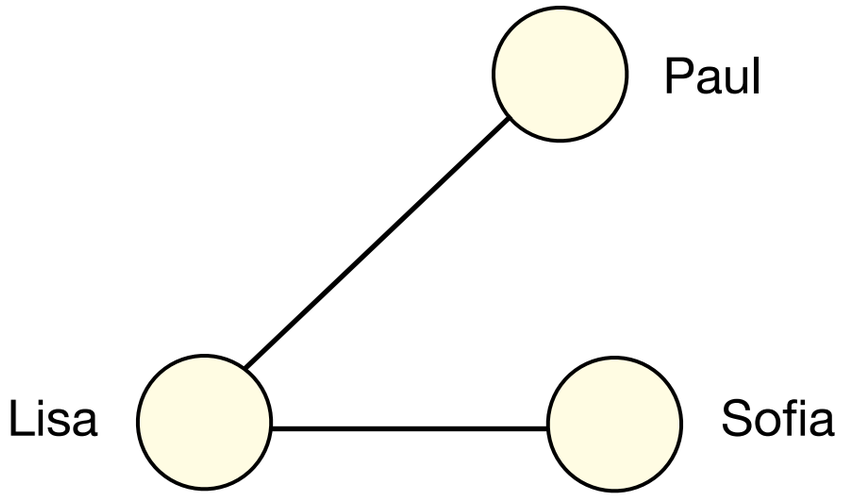


Derek Greene

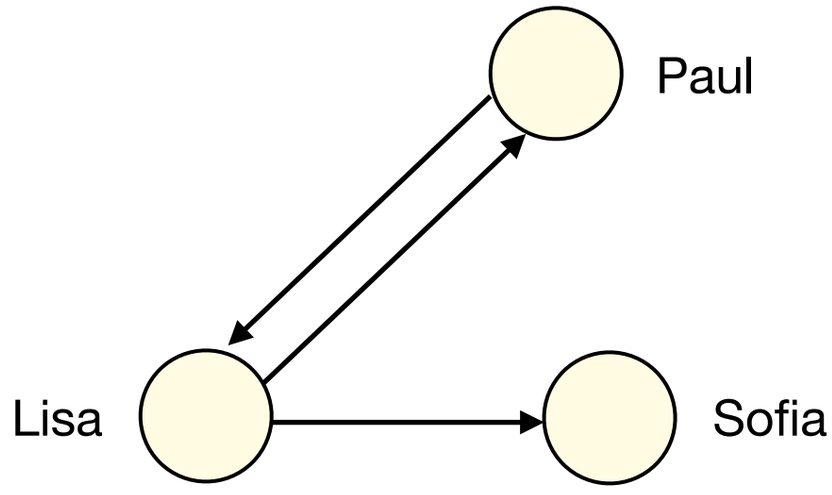
UCD School of Computer Science

Spring 2026

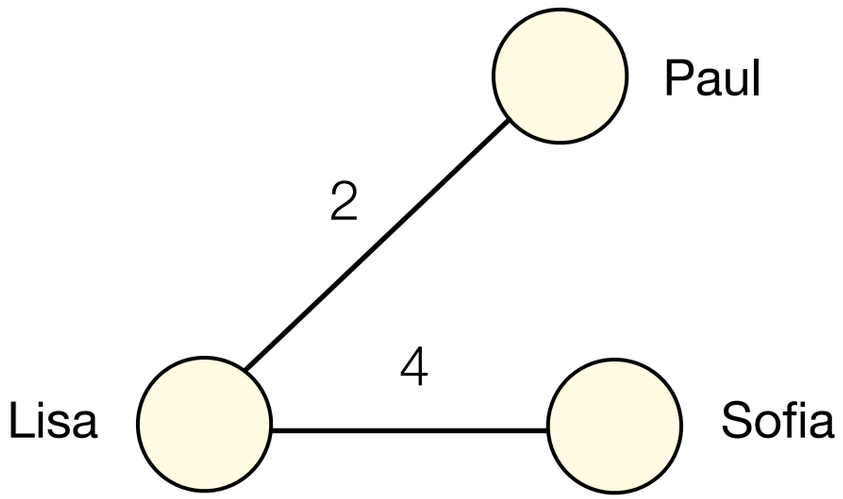
Reminder: Types of Networks



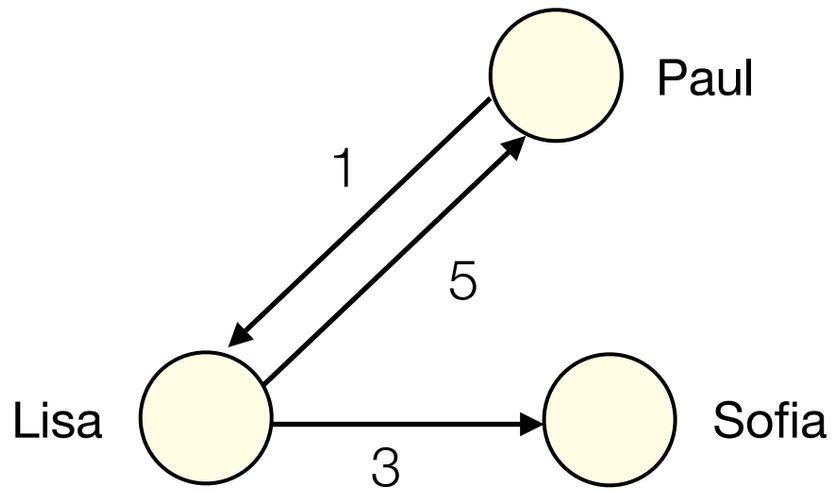
Undirected unweighted network



Directed unweighted network



Undirected weighted network



Directed weighted network

Creating Networks

- A dataset can be represented as a network whenever there are meaningful relationships between pairs of items or individuals. We follow a few general steps to construct a network:

1. **Decide on the network type.** Based on how the relationship between items or individuals works, choose whether the network should be directed or undirected.
2. **Define nodes with unique identifiers.** Each item or individual becomes a node. Every node must have a unique identifier so that it can be clearly distinguished from the others.
3. **List all pairs and add edges.** For every instance of a relationship in the data, connect the corresponding two nodes with an edge that represents that relationship.
4. **Verify the network.** Check that the network matches the original data - i.e., that all relevant items or individuals appear as nodes, all recorded pairwise relationships are present as edges, and no duplicate nodes or edges have been introduced.

Example: Creating Undirected Networks

- Location proximity information for six individuals, where the "lives near" relationship means that, for example, Alice lives within 5km of Robert and vice versa.

Raw Data

Person	Lives Near
Alice	Robert, Sharon
David	Elaine
Elaine	David
Keith	Robert, Sharon
Robert	Alice, Keith, Sharon
Sharon	Alice, Keith, Robert

Interpretation: "Alice lives near Robert and Sharon"

1. **Decide on the network type.** Because "lives near" is mutual, the data should be represented as an undirected network.
2. **Define nodes with unique identifiers.** Each of the six individuals becomes a node, where the node ID is their name.
3. **List all pairs and add edges.** For every pair of individuals who live near each other, we add a single undirected edge connecting their two nodes.
4. **Verify the network.** Check that the network matches the raw data.

Example: Creating Undirected Networks

- Location proximity information for six individuals, where the "lives near" relationship means that, for example, Alice lives within 5km of Robert and vice versa.

Person	Lives Near
Alice	Robert, Sharon
David	Elaine
Elaine	David
Keith	Robert, Sharon
Robert	Alice, Keith, Sharon
Sharon	Alice, Keith, Robert

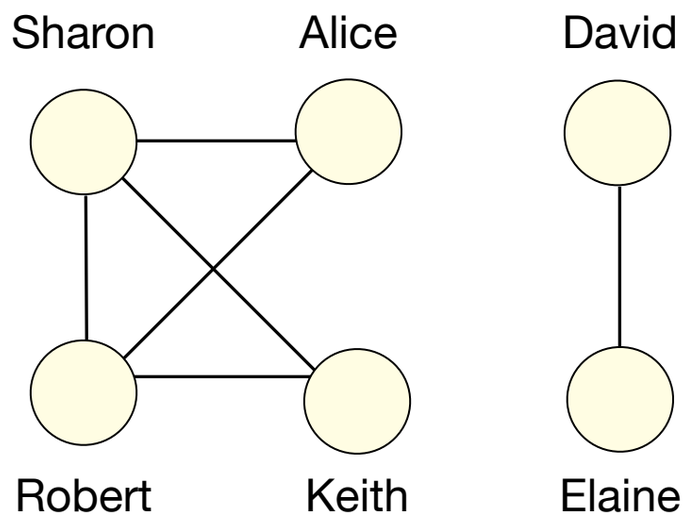
6 nodes with IDs:
Alice, David, Elaine,
Keith, Robert, Sharon

Find all unique pairs

(Alice, Robert)
(Alice, Sharon)
(David, Elaine)
(Keith, Robert)
(Keith, Sharon)
(Robert, Sharon)

6 unique pairs of individuals

Add edges from unique pairs



6 nodes, 6 undirected edges

Example: Creating Directed Networks

- Phone contact lists for five individuals, where the “has in contact lis” relationship is one-way - e.g. Alice can store Bob's number even if Bob does not store Alice’s number.

Raw Data

Person	Contact List
Alice	Bob, Eric, Kate
Bob	Eric
Dylan	Eric
Eric	Alice, Dylan
Kate	Alice, Bob

Interpretation:
"Alice has contacts Bob, Eric, and Kate"

1. **Decide on the network type.** Because “has in contact list” is not necessarily mutual, use a directed network.
2. **Define nodes with unique identifiers.** Each of the five individuals becomes a node, where the node ID is their name.
3. **List all pairs and add edges.** For every entry in a contact list, add a directed edge from the person who owns the phone to the person whose number is stored.
4. **Verify the network.** Check that the network matches the raw data.

Example: Creating Directed Networks

- Phone contact lists for five individuals, where the “has in contact lis” relationship is one-way - e.g. Alice can store Bob's number even if Bob does not store Alice’s number.

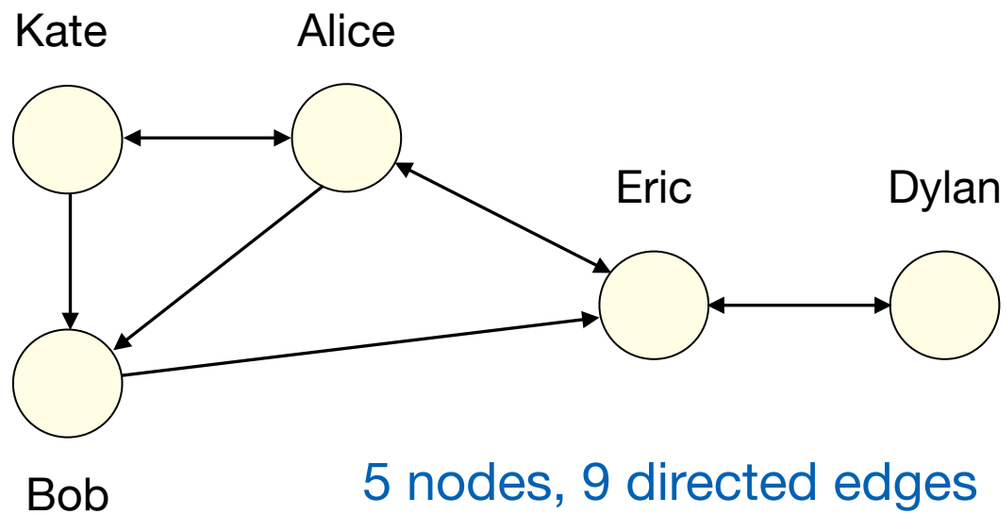
5 nodes with IDs: Alice, Bob, Dylan, Eric, Kate

Person	Contact List
Alice	Bob, Eric, Kate
Bob	Eric
Dylan	Eric
Eric	Alice, Dylan
Kate	Alice, Bob

Find all ordered pairs

(Alex, Eric)	(Eric, Dylan)
(Alex, Kate)	(Kate, Alex)
(Alex, Ruth)	(Kate, Ruth)
(Dylan, Eric)	(Ruth, Eric)
(Eric, Alex)	

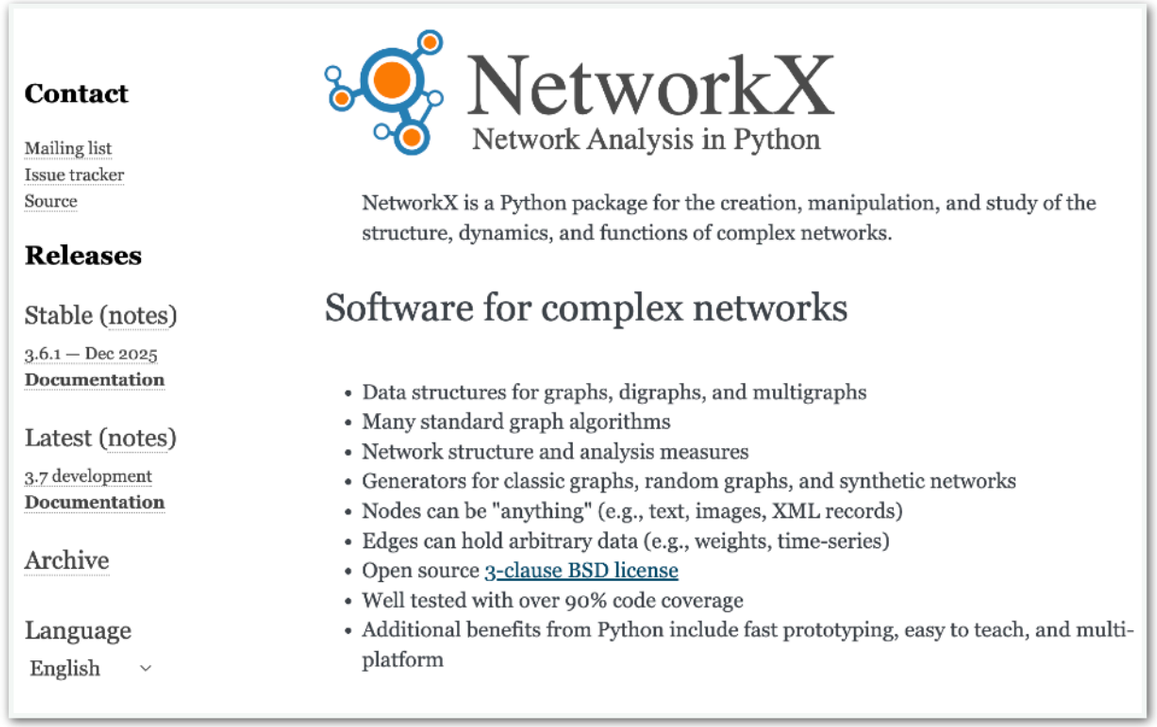
Add directed edges from the pairs



5 nodes, 9 directed edges

Network Analysis in Python

- In this module we will use the open source Python NetworkX package for working with networks: https://networkx.org



- Install version **3.6,** using PIP or Anaconda on the terminal or command line

```
> pip install networkx
```

```
> conda install networkx
```

- Then import the package to check the installation:

```
import networkx as nx
```

By convention we use the short alias `nx` to access the package

Creating Undirected Networks in Python

- To construct an empty **undirected network**, we create an instance of a NetworkX `Graph`.

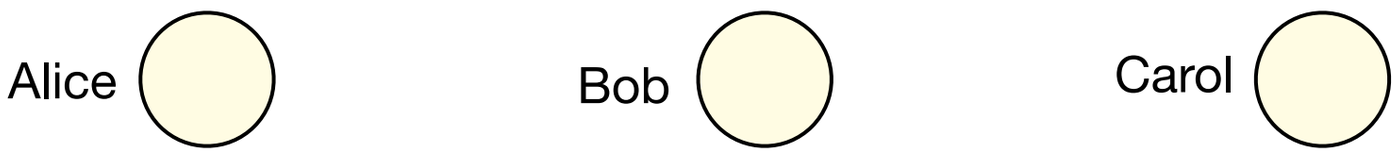
```
import networkx as nx
g = nx.Graph()
```

- There are several different ways to add nodes to a network.
- We can create new nodes one at a time using `add_node()`
- We give each node a unique identifier, typically a string or an integer. We can use this to refer to the node later.

```
g.add_node("Alice")
g.add_node("Bob")
g.add_node("Carol")
```

- We can also add new nodes in bulk using `add_nodes_from()`

```
names=["Alice", "Bob", "Carol"]
g.add_nodes_from(names)
```

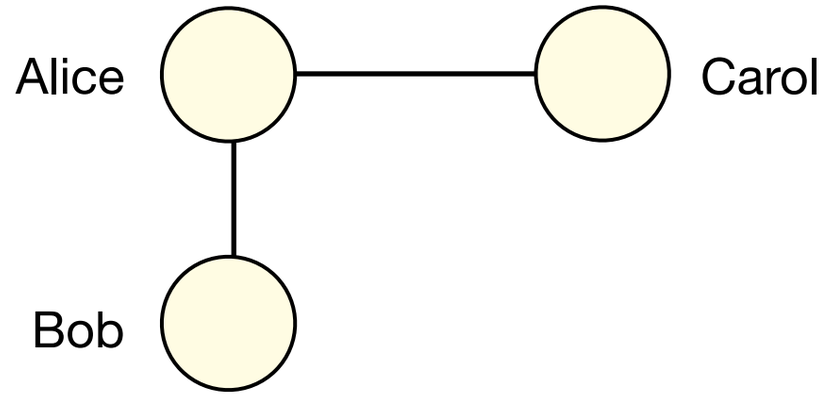


Creating Undirected Networks in Python

- There are several different ways to add new edges to a network with NetworkX.
- We can create edges one at a time by calling `add_edge()` and specifying the two relevant node identifiers as arguments.

```
g.add_edge("Alice", "Bob")
```

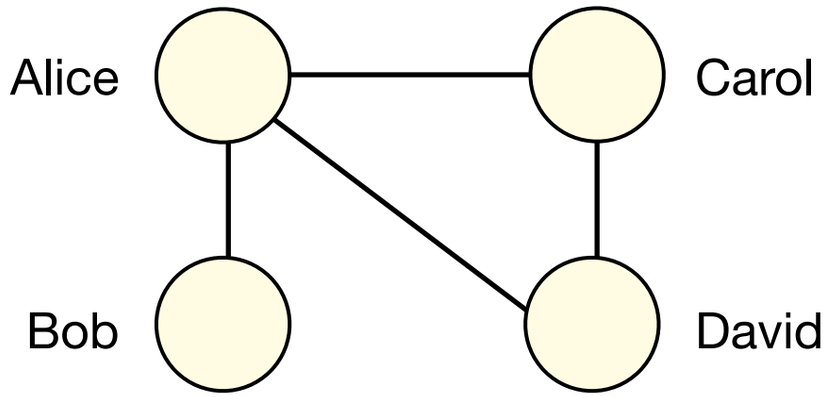
```
g.add_edge("Alice", "Carol")
```



- If nodes with the specified identifiers do not already exist, they will be added to the network.

```
g.add_edge("Alice", "David")
```

```
g.add_edge("Carol", "David")
```

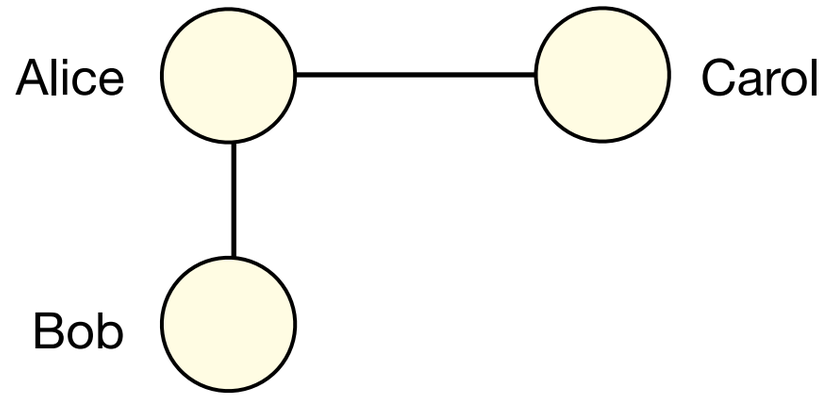


Creating Undirected Networks in Python

- There are several different ways to add new edges to a network with NetworkX.
- We can create edges one at a time by calling `add_edge()` and specifying the two relevant node identifiers as arguments.

```
g.add_edge("Alice", "Bob")
```

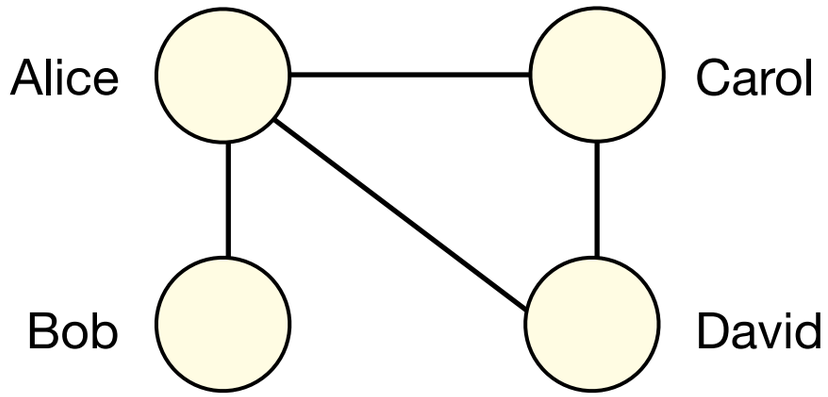
```
g.add_edge("Alice", "Carol")
```



- We can also add new edges in bulk using `add_edges_from()`

```
l = [("Alice", "David"),
     ("Carol", "David")]

g.add_edge_from(l)
```



Undirected Networks in Python

- Once we have created the network, we can check its size and contents.

```
g.number_of_nodes()
```

```
4
```

```
g.number_of_edges()
```

```
4
```

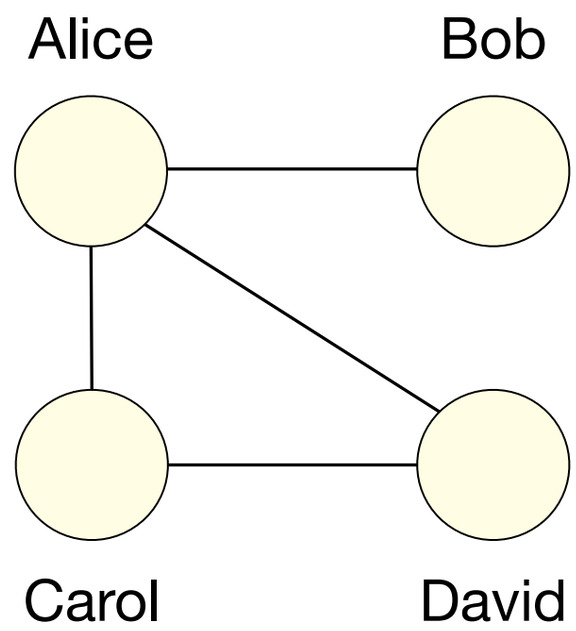
```
g.nodes()
```

```
NodeView(('Alice', 'Bob', 'Carol', 'David'))
```

```
g.edges()
```

```
EdgeView([('Alice', 'Bob'), ('Alice', 'Carol'),
('Alice', 'David'), ('Carol', 'David')])
```

```
g = nx.Graph()
g.add_node("Alice")
g.add_node("Bob")
g.add_node("Carol")
g.add_node("David")
g.add_edge("Alice", "Bob")
g.add_edge("Alice", "Carol")
g.add_edge("Alice", "David")
g.add_edge("Carol", "David")
```



Undirected Networks in Python

- **Example:** Location proximity data for six individuals. The mutual "lives near" relationship is represented using an undirected network.

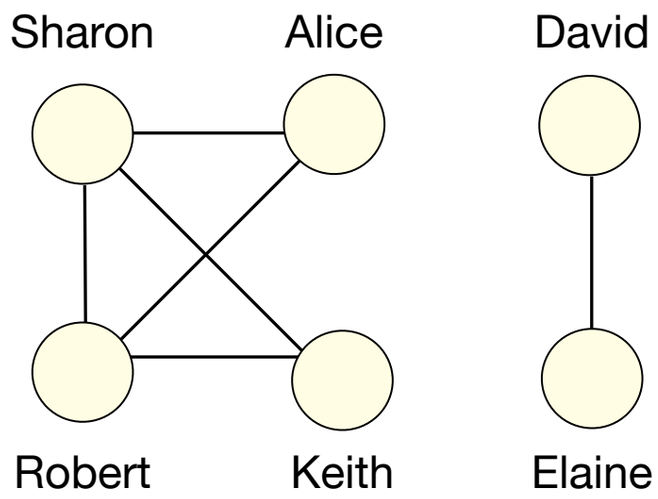
```
lives_near = {}
lives_near["Alice"] = ["Robert","Sharon"]
lives_near["David"] = ["Elaine"]
lives_near["Elaine"] = ["David"]
lives_near["Keith"] = ["Robert","Sharon"]
lives_near["Robert"] = ["Alice","Keith","Sharon"]
lives_near["Sharon"] = ["Alice","Keith","Robert"]
```

- Identify all pairs and add edges for those pairs. Note that nodes are created automatically, and duplicate edges are ignored.

```
g = nx.Graph()
for name1 in lives_near:
    for name2 in lives_near[name1]:
        g.add_edge(name1, name2)
```

```
g.number_of_nodes(), g.number_of_edges()
```

```
(6, 6)
```



6 nodes, 6 undirected edges

Directed Networks in Python

- To construct an empty **directed network**, we create an instance of a NetworkX `DiGraph`:
- We can add nodes and edges in a similar way as with an undirected network.
- However, the order of the two node identifiers passed to `add_edge()` is now important.

```
import networkx as nx
g = nx.DiGraph()
```

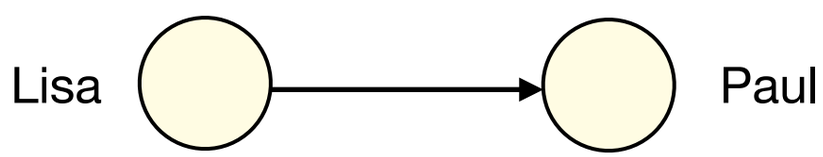
```
g.add_node("Lisa")
g.add_node("Paul")
g.add_node("Sofia")
```

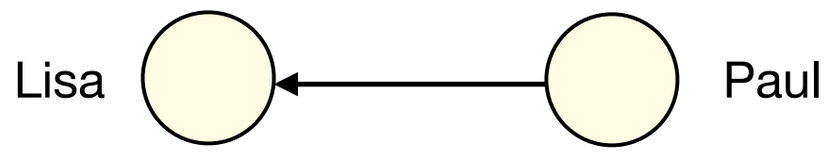
```
g.add_edge("Lisa", "Sofia")
g.add_edge("Lisa", "Paul")
g.add_edge("Paul", "Lisa")
```

```
g.add_edge("Lisa", "Paul")
```

!=

```
g.add_edge("Paul", "Lisa")
```





Directed Networks in Python

- Again we can check the network's size and contents...

```
g.number_of_nodes()
```

```
3
```

```
g.number_of_edges()
```

```
3
```

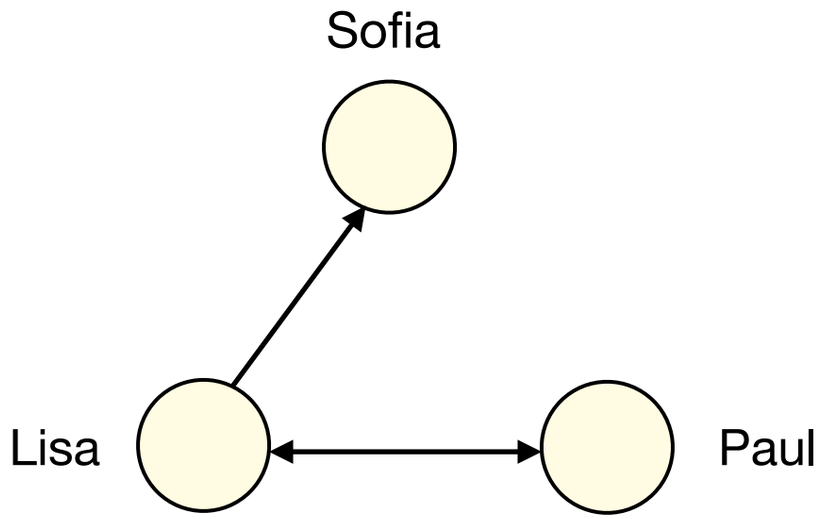
```
g.nodes()
```

```
NodeView(('Lisa', 'Paul', 'Sofia'))
```

```
g.edges()
```

```
OutEdgeView([('Lisa', 'Sofia'), ('Lisa',
'Paul'), ('Paul', 'Lisa')])
```

```
g = nx.DiGraph()
g.add_node("Lisa")
g.add_node("Paul")
g.add_node("Sofia")
g.add_edge("Lisa", "Sofia")
g.add_edge("Lisa", "Paul")
g.add_edge("Paul", "Lisa")
```



Edge between Lisa and Paul is reciprocal

Directed Networks in Python

- **Example:** Phone contact list data for five individuals. For this type of non-mutual relationship, we create a directed network.

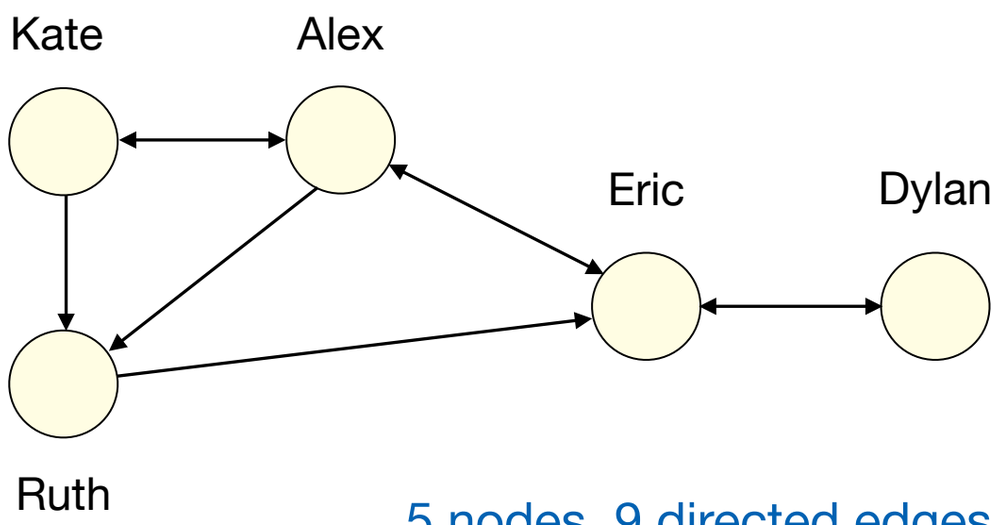
```
contacts = {}
contacts["Alex"] = ["Eric","Kate","Ruth"]
contacts["Dylan"] = ["Eric"]
contacts["Eric"] = ["Alex","Dylan"]
contacts["Kate"] = ["Alex","Ruth"]
contacts["Ruth"] = ["Eric"]
```

- Identify all pairs and create edges for those pairs. Note the order of the pair of node identifiers passed to `add_edge()` matters.

```
g = nx.DiGraph()
for name1 in contacts:
    for name2 in contacts[name1]:
        g.add_edge(name1, name2)
```

```
g.number_of_nodes(), g.number_of_edges()
```

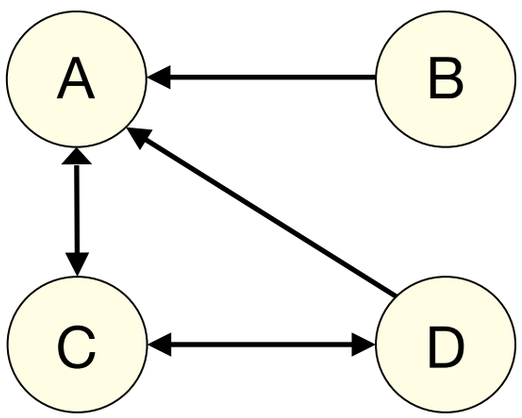
```
(5, 9)
```



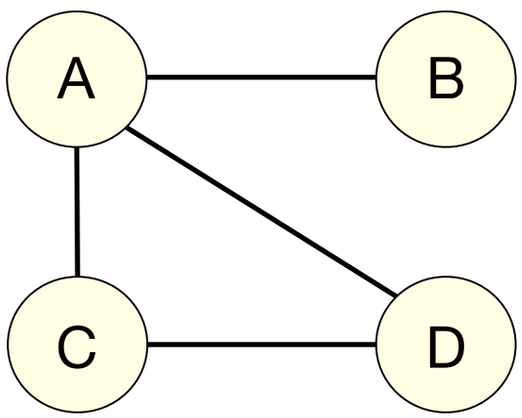
5 nodes, 9 directed edges

Converting Directed Networks

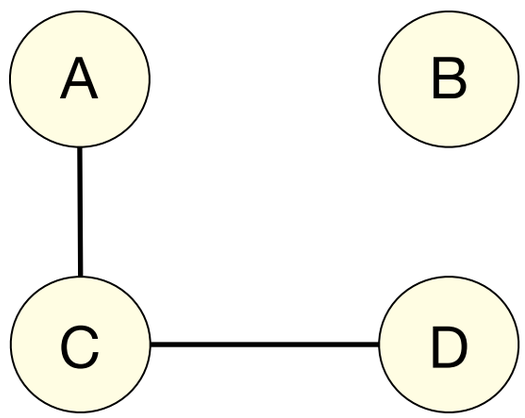
- In some instances it is useful to convert a directed network into an undirected one. For instance, some network analysis algorithms only work with undirected networks.
- Two common conversion strategies:
  1. **Union strategy:** If either edge $A \rightarrow B$ or $B \rightarrow A$ exists in the original directed network, then a single undirected edge between A and B is created.
  2. **Reciprocal-only strategy:** An undirected edge between A and B is created only if both directed edges $A \rightarrow B$ and $B \rightarrow A$ exist in the original directed network.



Original directed network



Undirected network 1



Undirected network 2

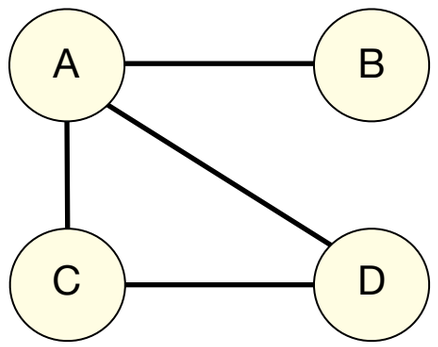
Converting Directed Networks

- In Network X, to convert a directed network to an undirected network, we use the `to_undirected()` method.
- The method accepts a `reciprocal` argument that determines which conversion strategy is used. A value `True` applies the reciprocal-only strategy, `False` applies the union strategy.

```
g1 = nx.DiGraph()
g1.add_edge("B","A")
g1.add_edge("A","C")
g1.add_edge("C","A")
g1.add_edge("C","D")
g1.add_edge("D","C")
g1.add_edge("D","A")
```

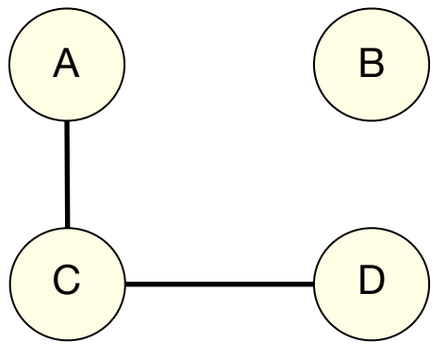
```
g2 = g1.to_undirected(False)
g2.edges()
```

```
EdgeView([('B', 'A'), ('A', 'C'), ('A', 'D'), ('C', 'D')])
```




```
g3 = g1.to_undirected(True)
g3.edges()
```

```
EdgeView([('A', 'C'), ('C', 'D')])
```



Attributed Networks

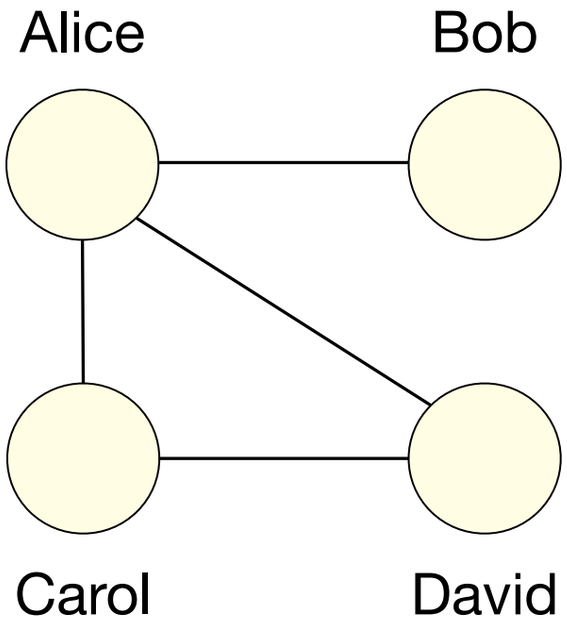
- In many applications we will have additional information about nodes or edges, beyond simply whether two nodes are connected.

Network	Node-Related Metadata	Edge-Related Metadata
Transaction network	Customer name; customer segment; address	Transaction amount; timestamp; currency
Flight network	Airport code; city; country	Flight distance; timestamp; operating airline
Email network	Full name; department/ team; role; location	Timestamp; subject; number of attachments
Social media network	Account type; follower count; location	Interaction type; interaction count; text content

Node Attributes

- For some networks, each node will have additional metadata associated with it, referred to as the node's attributes.
- The exact attributes depend on the nature of the data.

Network where each node has 3 attributes, in addition to the node's ID: {Gender, Age, Location}

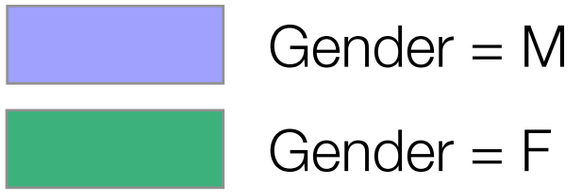


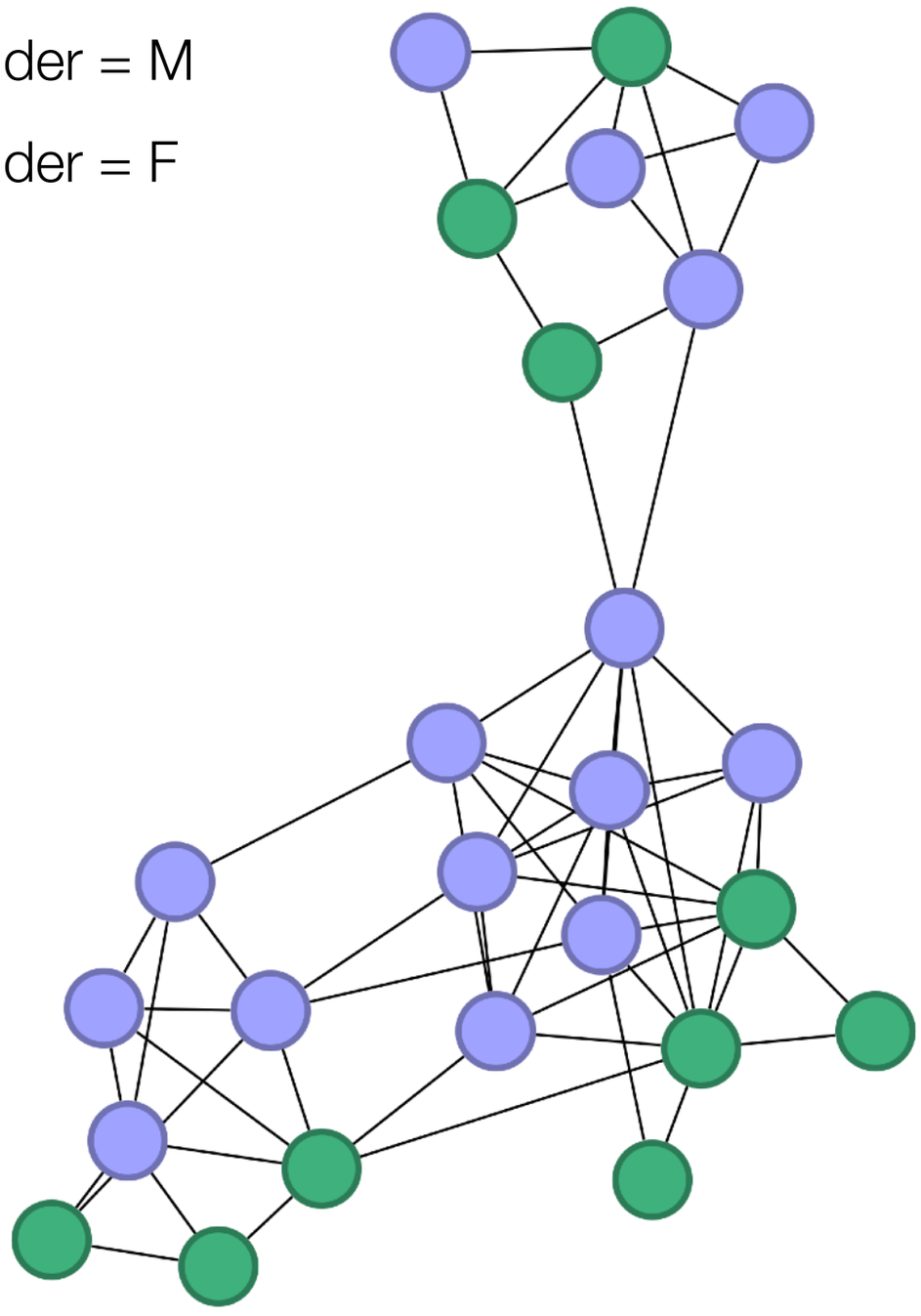
Node ID	Gender	Age	Location
Alice	F	21	Galway
Bob	M	19	Dublin
Carol	F	23	Cork
David	M	20	Galway

Example: Node Attributes

- **Example:** Network of LinkedIn connections, where each node has an attribute "Gender".

Node ID	Gender
Alan Dent	M
Alex Mitchell	M
Alice Muzhikov	F
Alison Gayle	F
Barry Patterson	M
Bob O'Gara	M
Claire Scott	F
David Lennon	M
Eric Smith	M
Hugh Wright	M
...	...

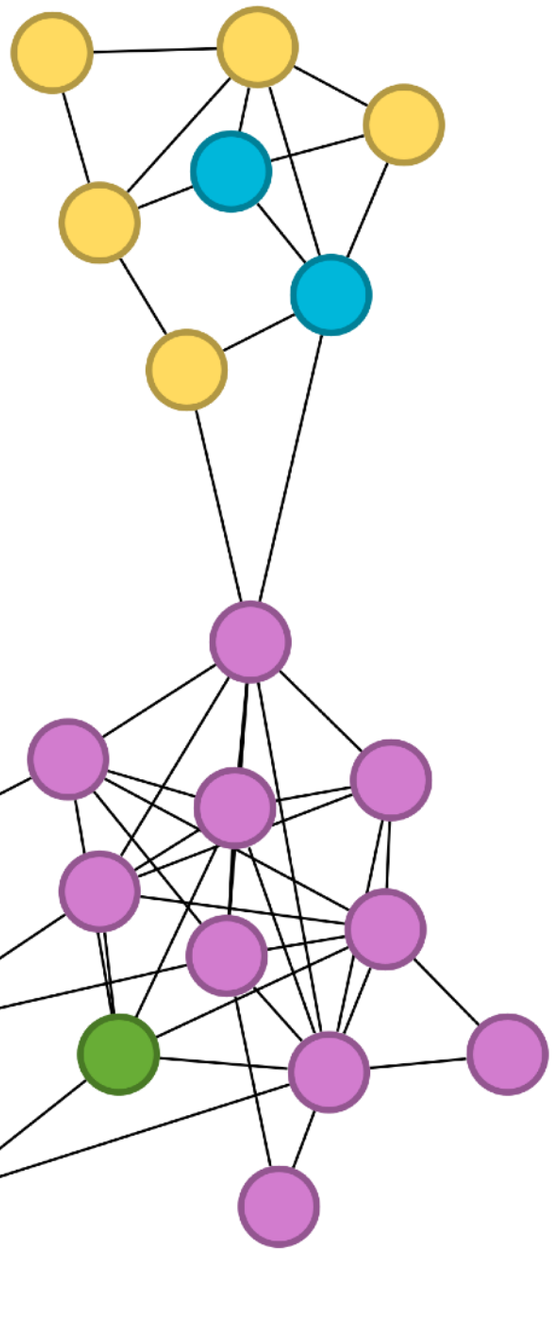




Example: Node Attributes

- **Example:** Network of LinkedIn connections, where each node has an attribute "Industry".

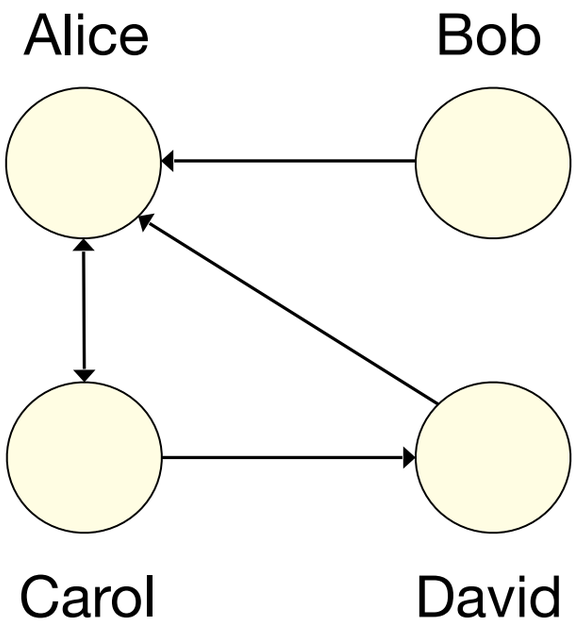
Node ID	Industry
Alan Dent	Finance
Alex Mitchell	IT
Alice Muzhikov	Finance
Alison Gayle	Retail
Barry Patterson	IT
Bob O'Gara	Recruitment
Claire Scott	Retail
David Lennon	Recruitment
Eric Smith	IT
Hugh Wright	IT
...	...



Edge Attributes

- In some networks, edges will also have attributes associated with them. These attributes provide additional information about the nature of the connections between nodes.

Directed network where each edge has 3 attributes:
{Timestamp, Currency, Amount}



Node 1	Node 2	Timestamp	Currency	Amount
Bob	Alice	23/11/2025 11:34	Euro	1050.00
Alice	Carol	06/11/2025 18:04	Euro	825.00
Carol	Alice	22/11/2025 09:33	Euro	4320.50
David	Alice	19/12/2025 13:40	GBP	1900.00
Carol	David	11/01/2026 10:08	Euro	786.40

Attributes in NetworkX

- In NetworkX, any Python value can be added as an attribute of a node or an edge. Each node and edge stores its attributes in a dictionary of key-value pairs.
- We can set attribute values when calling the `add_node()` method. We do not need to predefine all possible attributes in advance.

```
g.add_node(1, label="Ireland", population=5.4)
g.add_node(2, label="Spain", population=49.4)
g.add_node(3, label="Italy", capital="Rome")
```

- Alternatively, we can use `g.nodes` like a dictionary to set or modify attribute values for an existing node:

```
g.nodes[1]["capital"] = "Dublin"
g.nodes[2]["capital"] = "Madrid"
```

```
g.nodes[1]["population"] += 0.2
```

Attributes in NetworkX

- Similarly, we can set edge attribute values when adding an edge:

```
g.add_edge("Alice", "Bob", amount=1050.0, currency="euro")
g.add_edge("Alice", "Carol", amount=825.0, currency="euro")
g.add_edge("David", "Alice", amount=1900.0, currency="gbp")
```

- Once we have created the network, we can access the edge attributes in the network in various ways.

```
g.edges(data=True)
```

```
OutEdgeDataView([('Alice', 'Bob', {'amount': 1050.0, 'currency': 'euro'}),
('Alice', 'Carol', {'amount': 825.0, 'currency': 'euro'}), ('David', 'Alice',
{'amount': 1900.0, 'currency': 'gbp'})])
```

```
g["Alice"]
```

```
AtlasView({'Bob': {'amount': 1050.0, 'currency': 'euro'},
'Carol': {'amount': 825.0, 'currency': 'euro'}})
```

Access all edge data for node *Alice*

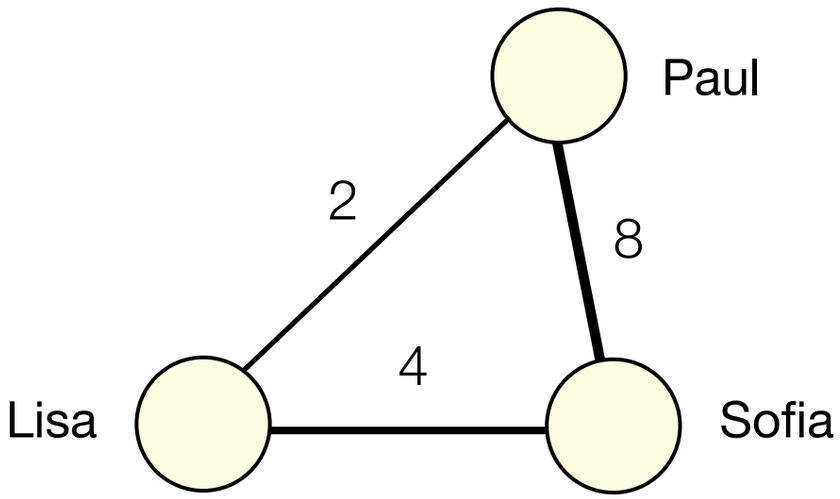
```
g["Alice"]["Carol"]
```

```
{'amount': 825.0, 'currency': 'euro'}
```

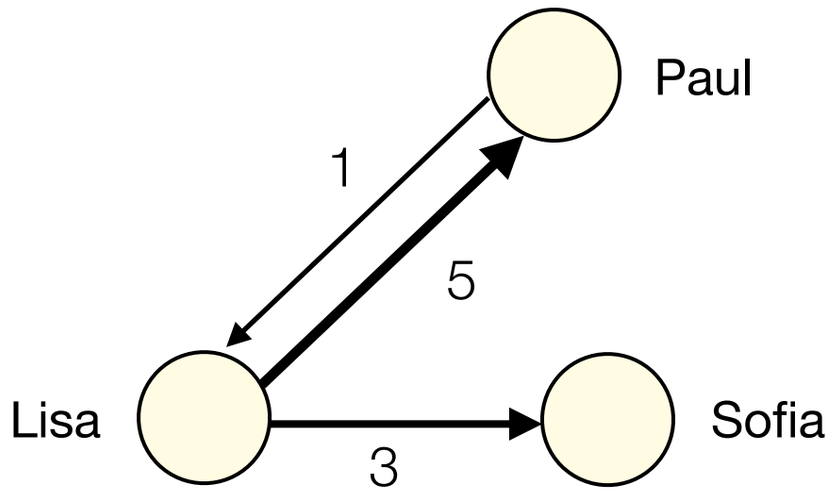
Access attributes for a single edge

Reminder: Weighted Networks

- In some networks, all edges have equal importance. These are called unweighted networks.
- In other situations, each edge has a weight value that indicates the strength or frequency of the connection between two nodes.
- Weighted networks can be used to represent both mutual and non-mutual relationships between items or individuals.



Undirected weighted network



Directed weighted network

Creating Weighted Networks

- When relationships can occur multiple times or with varying strength, we need to adapt the basic network creation steps to capture this.

1. **Decide on the network type.** Choose whether the network should be directed or undirected, and whether repeated interactions between the same pair should be captured as edge weights.
2. **Define nodes with unique identifiers.** Each item or individual becomes a node. Every node must have a unique identifier so that it can be clearly distinguished from the others.
3. **Add weighted edges.** For every instance of a relationship in the data, connect the corresponding two nodes. Add an appropriate numeric weight to the edge based on the strength or frequency of the connection between them.
4. **Verify the network.** Check that it matches the raw data, including that edge weights correctly reflect the frequency of each pair.

Example: Creating Weighted Networks

- **Example:** Team meeting attendance data can be modelled as a weighted network, where the weight on an edge represents how many meetings a pair of individuals attended together.

Raw meeting attendance data

Day	Meeting Attendees
Monday	Lisa, Paul
Tuesday	Lisa, Paul
Wednesday	Lisa, Sofia
Thursday	David, Lisa, Sofia
Friday	Alice, David, Lisa, Sofia
Saturday	Alice, Carol

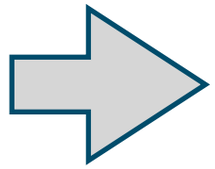
Interpretation:
"Lisa and Paul both attended Monday's team meeting"

1. **Decide on the network type.** Because "attended the same meeting" is mutual, use an undirected network.
2. **Define nodes with unique identifiers.** Each of the six individuals becomes a node, where the node ID is their name.
3. **Add weighted edges.** For every meeting, list all pairs of attendees. For each pair, add or update the edge whose weight is the number of meetings that pair attended together.
4. **Verify the network.** Check that the network matches the raw data.

Example: Creating Weighted Networks

- **Example:** Team meeting attendance data can be modelled as a weighted network, where the weight on an edge represents how many meetings a pair of individuals attended together.

Day	Meeting Attendees
Monday	Lisa, Paul
Tuesday	Lisa, Paul
Wednesday	Lisa, Sofia
Thursday	David, Lisa, Sofia
Friday	Alice, David, Lisa, Sofia
Saturday	Alice, Carol



Find the counts for each unique pair

(Alice, Carol)	1
(Alice, David)	1
(Alice, Lisa)	1
(Alice, Sofia)	1
(David, Lisa)	2
(David, Sofia)	2
(Lisa, Paul)	2
(Lisa, Sofia)	3

Counts for 8 unique pairs

6 nodes with IDs: Alice, Carol, David, Lisa, Paul, Sofia

Extract unordered pairs for each meeting
e.g. Thursday: (David, Lisa), (David, Sofia), (Lisa, Sofia)

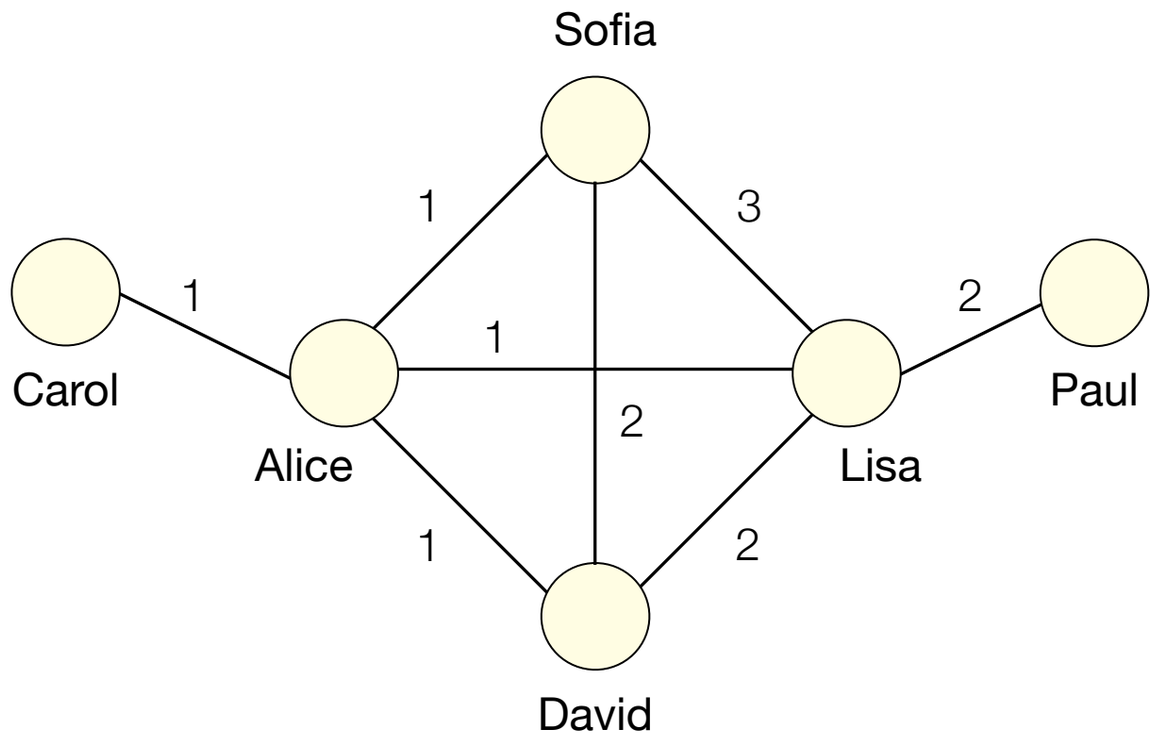
Example: Creating Weighted Networks

- **Example:** Team meeting attendance data can be modelled as a weighted network, where the weight on an edge represents how many meetings a pair of individuals attended together.

(Alice, Carol)	1
(Alice, David)	1
(Alice, Lisa)	1
(Alice, Sofia)	1
(David, Lisa)	2
(David, Sofia)	2
(Lisa, Paul)	2
(Lisa, Sofia)	3

Counts for 8 unique pairs

Add weighted edges from pair counts

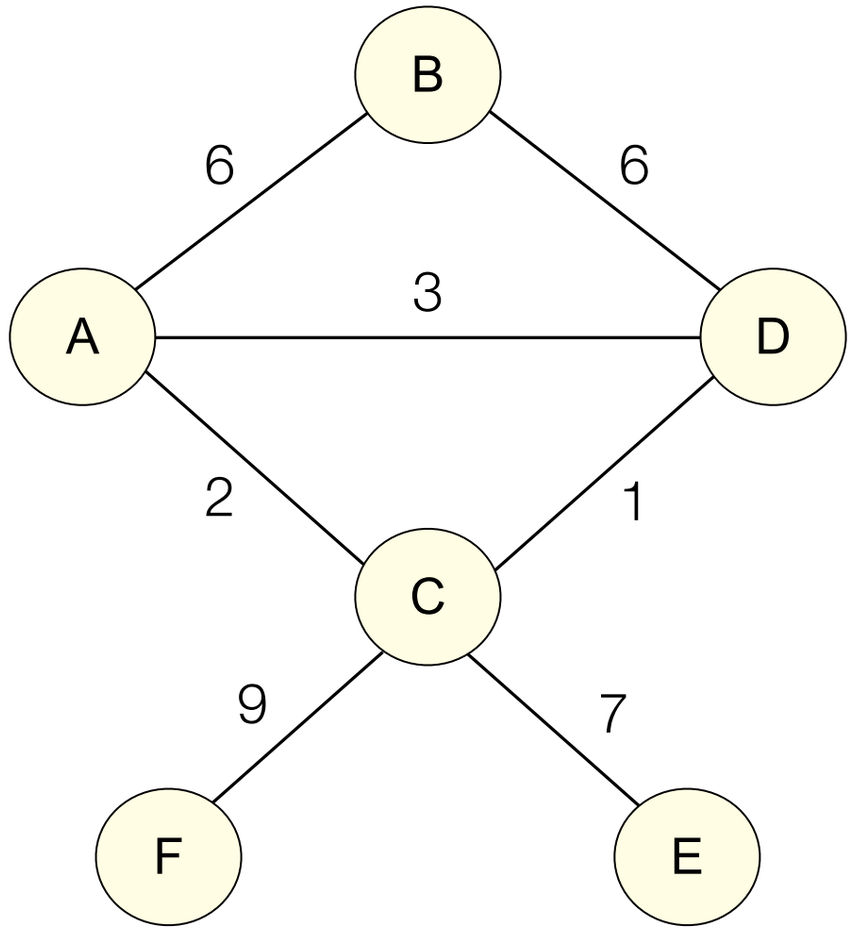


6 nodes, 8 weighted undirected edges

Weighted Networks in Python

- In NetworkX, we create weighted networks by storing a numeric value in a special edge attribute called `weight`. These weights can be either integers or floats.
- **Example:** Create an undirected network with 6 nodes and 7 edges, where each edge has an integer weight.

```
g = nx.Graph()
g.add_edge("A", "B", weight=6)
g.add_edge("A", "C", weight=2)
g.add_edge("B", "D", weight=6)
g.add_edge("C", "D", weight=1)
g.add_edge("C", "E", weight=7)
g.add_edge("C", "F", weight=9)
g.add_edge("A", "D", weight=3)
```



Weighted Networks in Python

- After the weighted network has been created, edge weights can be accessed and modified like any other edge attribute by referring to the attribute named `weight`.

```
g["A"]["B"]["weight"]
```

```
6
```

```
for y in g.edges(data=True):
  print(y)
```

```
('A', 'B', {'weight': 6})
('A', 'C', {'weight': 2})
('A', 'D', {'weight': 3})
('B', 'D', {'weight': 6})
('C', 'D', {'weight': 1})
('C', 'E', {'weight': 7})
('C', 'F', {'weight': 9})
```

```
g["A"]["B"]["weight"] += 1
```